

ACTIVIDAD 5.1 – Informes WPF & Android

Objetivos

Integrar la generación de informes **directamente dentro** de la aplicación que ya tienes (sin salir de ella) aplicando los conceptos del Tema 5.1 - Confección de informes. Con ello se mejora la UX ofreciendo al usuario un botón que genera y visualiza/imprime el informe con un solo clic.

Parte A – WPF: Informe RDLC desde el “Formulario de Empleados”

Qué debe hacer el informe:

- Listado de todos los empleados (Nombre, Apellidos, DNI, E-mail, Teléfono).
- Cabecera con logo de la empresa y título “Informe de Personal – [Fecha]”.
- Total de empleados.

Desarrollo

1. Utiliza el proyecto WPF (Formulario de Empleados).
2. Añade el control **ReportViewer** al formulario principal (desde la caja de herramientas → Reporting).
3. Crea el informe: Proyecto → Agregar → Nuevo elemento → **Report (RDLC)** → Nombre: InformeEmpleados.rdlc.
 - Configura el origen de datos usando el mismo DataSet o lista que ya estás usando en el formulario (clase Empleado).
4. Diseña el informe en el modo Design:
 - Arrastra una Tabla y añade los campos que ya existen (Nombre, Apellidos, DNI, E-mail, Teléfono).
 - En el pie de tabla pon la expresión:
 - Total empleados: =CountRows("DataSet1")
5. Crea el botón “Generar Informe” (en el evento Click).

Entrega: Crea un pdf añadiendo la captura de pantalla del informe generado, captura del interfaz de la aplicación, el código del botón “Generar informe” y el archivo .rdlc exportado.

PARTE B – Android (Kotlin): Informe PDF desde la app del “Ejercicio 6 Android - Navegación entre ventanas” con iText7.

Qué debe hacer el informe:

- PDF con título “Informe de Android” (en negrita).
- Mostrar el mensaje de saludo mostrado en la segunda pantalla.
- Fecha de generación del informe (dd/MM/yyyy).
- Botón “Generar PDF” que crea el archivo y lo abre automáticamente.

Desarrollo:

1. En tu proyecto Android (navegación entre ventanas) abre el módulo app/build.gradle.kts y añade la dependencia (implementation("com.itextpdf:itext7-core:8.0.5"))
2. Crea el método para crear el documento en Kotlin con toda la configuración del informe:

```
// Ruta donde se guardará el archivo
val ruta = getExternalFilesDir(null)?.absolutePath + "/InformeSaludo.pdf"
val writer = PdfWriter(ruta)
val pdf = PdfDocument(writer)
val document = Document(pdf)

// 1. Título solicitado
document.add(Paragraph("Informe de Usuario")
    .setFontSize(22f)
    .setBold()
    .setTextAlignment(TextAlignment.CENTER))

// 2. Mensaje de saludo (el que ya tienes en el TextView)
document.add(Paragraph("\nContenido del saludo:").setItalic())
document.add(Paragraph(textoSaludo).setFontSize(16f))

// 3. Fecha de generación
val fecha = java.text.SimpleDateFormat("dd/MM/yyyy").format(java.util.Date())
document.add(Paragraph("\nFecha de generación: $fecha")
    .setTextAlignment(TextAlignment.RIGHT))

document.close()
Toast.makeText(this, "PDF generado con éxito", Toast.LENGTH_SHORT).show()

} catch (e: Exception) {
    Toast.makeText(this, "Error: ${e.message}", Toast.LENGTH_LONG).show()
}
```

Ejemplo



3. Añade un botón para generar el informe:

```
binding.btnGenerarPDF.setOnClickListener {  
    // Usas la variable 'saludo' que ya creaste arriba en tu código  
    generarInformePDF(saludo)  
}
```

Ejemplo

Nota: Asegúrate de tener configurado el FileProvider en el AndroidManifest.xml (explicado en la teoría).

Entrega: Continúa el PDF del apartado A (indica bien cada parte) y añade la Captura del PDF abierto en el móvil, el código de la función generarInformePDF(). Y la captura del build.gradle.kts con la dependencia.

Añade una portada y súbelo a MOODLE.